CSCE 221 Cover Page

Justus Languell 135007144

justus justus@tamu.edu

Please list all sources in the table below - including any websites or resources used as learning aids for the logic required to solve the question(s). If you fail to cite sources you can get a lower number of points or even zero.

If you are caught abusing LLMs or GenAI such as ChatGPT, Claude, etc... or services like Chegg or CourseHero to come up with solutions and or code that you did not create yourself, we are obligated to (and will) file a report to the Aggie Honor System Office with no exceptions. For a list of more details and potential sanctions read more on the honor code site here.

Type of Source			
Websites (provide URL)	tex.stackexchange.com	stackoverflow.com	
Printed Material			
Other Sources			

I certify that I have listed all the sources that I used to develop the solutions/codes to the submitted work.

"*On my honor as an Aggie, I have neither given nor received any unauthorized help on this academic work.*"

Justus Languell April 25, 2025

Homework 3

See the Canvas calendar for the deadline

Typeset your solutions to the homework problems preferably in LaTeX or LyX. See the class webpage for information about their installation and tutorials. There are 6 problems on 6 separate pages.

1. (10 points) An airport is developing a computer simulation of air-traffic control that handles events such as landings and takeoffs. Each event has a *timestamp* that denotes the time when the event occurs. The simulation program needs to **efficiently**[1] perform the following two fundamental operations:

   - Insert an event with a given timestamp (that is, add a future event)
   - Extract the event with a **smallest** timestamp (that is, determine the next event to process)

   (a) What data structure should be used to implement the above operations **efficiently**? Explain your reasoning.

   I would implement this system using a minimum priority queue implemented with a min-heap.

   (b) Provide the big-O asymptotic complexity of inserting a timestamp into the data structure identified in part 1a

   The time complexity of inserting a timestamp into this min-heap is $O(log \cdot n)$.

   (c) Provide the big-O asymptotic complexity of extracting a timestamp from the data structure identified in part 1a

   The time complexity of extracting the smallest timestamp from the min-heap is $O(1)$.

[1] Refers to the best correctly performing data structure (covered in this course) on grounds of big-O time complexity per type of operation; i.e. "$O(1)$ push, $O(logn)$ pop is better than $O(n)$ push, $O(1)$ pop."

2. (15 points) A *complete* graph is an **undirected** graph in which every pair of vertices are connected with an edge. Consider the following complete graph with $n = 6$ vertices.

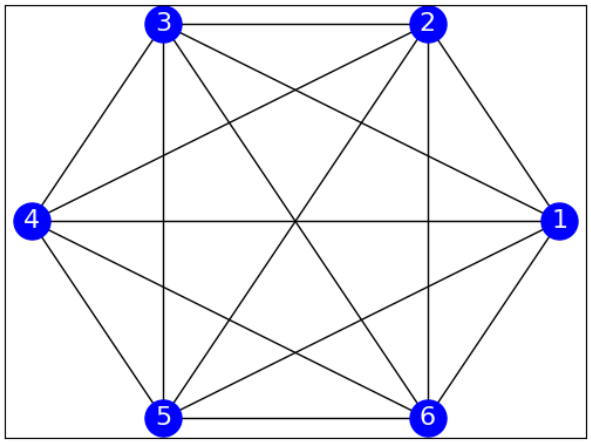


Figure 1: A complete graph with $n = 6$ vertices

(a) In what order does `DFS` explore vertices in the above graph? Assume `DFS` starts at vertex 4. The adjacency lists are in ascending order by the numeric label.

The ascending adjacency list starting at vertex 4 is $4 : 1, 2, 3, 5, 6$. Therefore the DFS traversal followed that order: $4 \to 1 \to 2 \to 3 \to 5 \to 6$.

(b) What is the running time of `DFS` on a complete graph with n vertices? Provide an asymptotic Big-O bound in terms of the number of vertices. Explain your reasoning.

The time complexity of DFS is expressed as $O(E + V)$ where E is the number of edges and V is the number of vertices. In the graph there are $V = n$ vertices, and since the graph is strongly connected, there is an edge connecting each of the n vertices to each $n - 1$ other vertices. Given undirected edges we have half the number of edges because there need not be duplicate directions, so the total number of edges is $E = \frac{n(n-1)}{n}$. This yields an overall time complexity of $O(n + \frac{n(n-1)}{2})$ which resolves to $O(n^2)$.

(c) How many back edges, forward edges, cross edges, and exploratory edges are generated by running `DFS` on a complete graph with n vertices?

Recall the total number number of edges $E = \frac{n(n-1)}{2}$. Firstly, we know that to visit all n vertices from an initial vertex, we must use $n - 1$ edges as exploratory because there are $n - 1$ edges connecting n nodes. The remaining edges are $\frac{n(n-1)}{2} - (n - 1) = \frac{(n-1)(n-2)}{2}$ are all forward edges, as there are no cross edges.

- Exploratory edges $E = n - 1$
- Back edges $E = \frac{n(n-1)}{2}$
- Forward edges $E = \frac{(n-1)(n-2)}{2}$
- Cross edges $E = 0$

3. (10 points) Answer each of the following questions with a tight big-O asymptotic bound. Justify with algorithmic reasoning.

   (a) A priority queue, `UnsortedMPQ`, is implemented based on an **unsorted array**. In terms of n, the number of elements, what is the big-O runtime of the operation which retrieves the minimum value?

   $$\texttt{unsorted_min(n)} \in O(\ldots)$$

   We are told that this priority queue ADT is implemented with an unsorted array, rather than a min-heap which uses an unsorted array, so it seems that the algorithm would need an $O(n)$ pass to scan for the minimum element.

   (b) Dijkstra's algorithm is implemented based on this unsorted minimum priority queue. In terms of v and e (vertices, edges) what is the big-O runtime of this implementation of Dijkstra's algorithm?

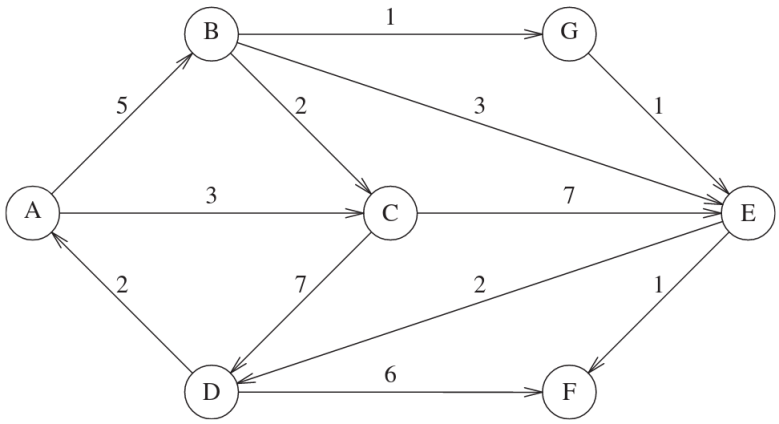
   $$\texttt{dijkstra(v,e)} \in O(\ldots)$$

   To find the total time complexity of Dijkstra's algorithm we need to find the time complexity of each component in terms of v and e.

   - Adding all the vertices into this priority queue is just appending them all to an array, which is $O(v)$ time complexity.
   - For each of the v vertices we need to find the minimum of this min-heap, which is $O(v)$ for each as we saw in (a), therefore the total time complexity of this step is $O(v \cdot v) = O(v^2)$.
   - Relaxing the edges is runtime $O(e)$.

   So when we combine the terms, we get the time complexity $O(v + v^2 + e) = O(v^2 + e)$.

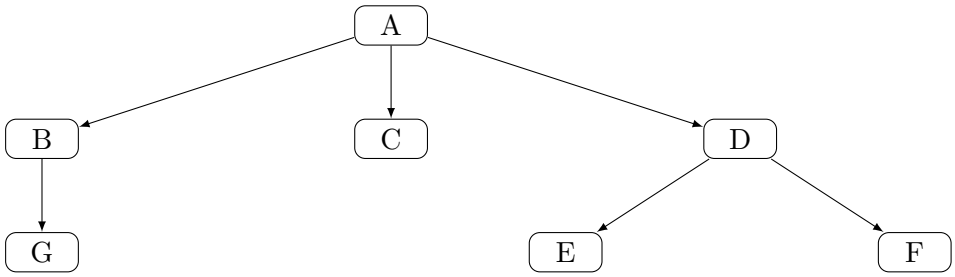
4. (15 points) Find the shortest path from D to all other vertices for the graph below.



(a) Illustrate[2] the minimum priority queue at each iteration of Dijkstra's algorithm.

0: $(A, 0)$
1. $(D, 2), (C, 3), (B, 5)$
2. $(C, 3), (E, 4), (B, 5), (F, 8)$
3. $(E, 4), (B, 5), (F, 8)$
4. $(B, 5), (F, 8)$
5. $(G, 6), (F, 8)$
6. $(F, 8)$
7.

(b) Draw the Shortest Path Tree.

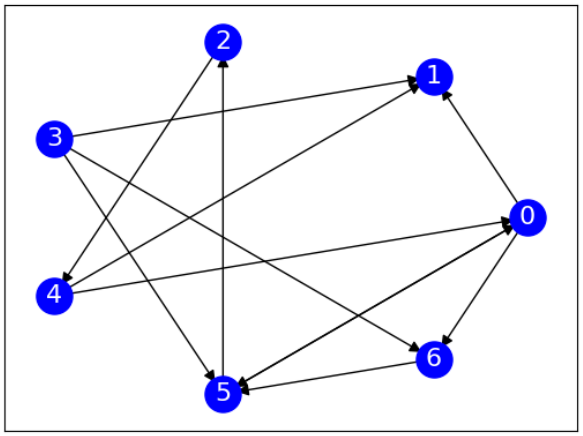


(c) What is the big-O time complexity of Dijkstra's algorithm in terms of v, e under the assumption that the graph is implemented based on an **adjacency list** and the minimum priority queue is implemented based on a **binary heap**?

$$O(v \cdot log(v) + e \cdot log(e))$$

[2]Provide a table, visual, or other demonstration of the minimum priority queue used in Dijkstra's progressing as the algorithm iterates, i.e. relaxations & key updates. In lecture & labs the same table is often used - feel free to use this one.

5. (15 points) Find the **shortest path** from vertex 3 to all other vertices for the graph below. Be careful when looking at what connections exist in this directed graph. Many students don't see that there is 1 edge from 0 to 5 and another edge from 5 to 0.



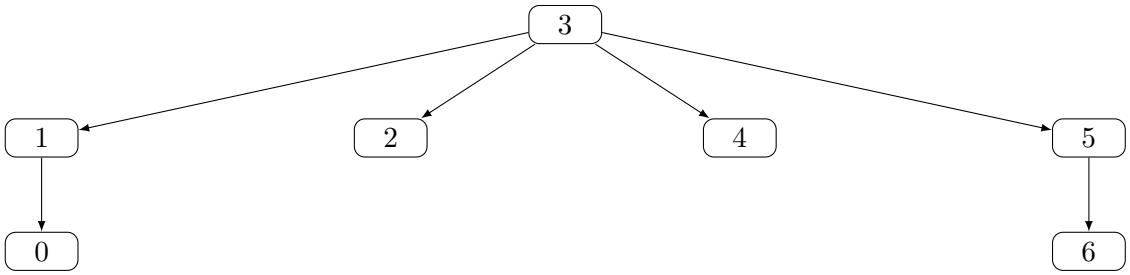
(a) Which graph algorithm can solve the problem most *efficiently*[3]?

Given no edge weights, the most efficient algorithm is a breadth-first search.

(b) How could you use the same algorithm if the graph had edge weights? (*Hint*: You may want to create intermediate nodes.)

Given edge weights, the most efficient algorithm is a Dijkstra's algorithm.

(c) Draw the Shortest Path Tree.



[3]Refers to the best correctly performing graph algorithm (covered in this course) on grounds of mainly big-O, and to some extent memory efficiency.

6. (15 points) There are five small islands in a lake, and the state wants to build four bridges to connect them so that each island can be reached from any other one via one or more bridges. The cost of bridge construction is proportional to its length. The distance between pairs of islands is given in the following table.

	1	2	3	4	5
1	-	10	15	10	20
2	-	-	15	20	20
3	-	-	-	15	30
4	-	-	-	-	10
5	-	-	-	-	-

Table 1: The distance between any two islands

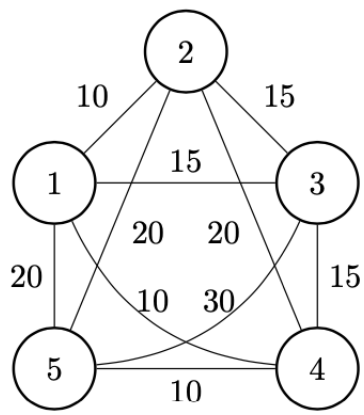


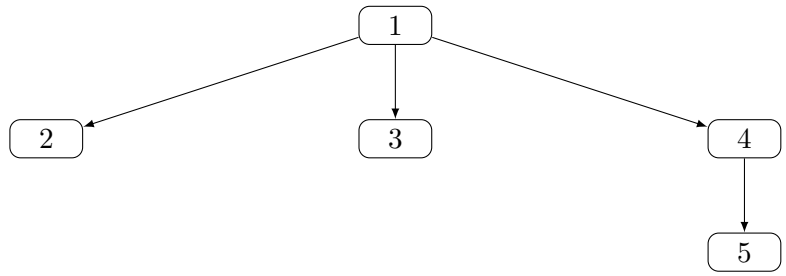
Figure 2: The distance between any two islands

(a) Illustrate the steps of Prim's algorithm using the graph above. Draw the Minimum Spanning Tree generated by your illustration. What is the length of the bridges?

Let (v, u, w) represented an edge from v to u with weight w. Let E be the set of available edges and V be the set of visited vertices.

- Begin at vertex 1, $V = \{1\}$,
  $E = \{(1, 2, 10), (1, 3, 15), (1, 4, 10), (1, 5, 20)\}$
- Select $(1, 2, 10)$, $V = \{1, 2\}$,
  $E = \{(1, 3, 15), (1, 4, 10), (1, 5, 20), (2, 3, 15), (2, 4, 20), (2, 5, 20)\}$
- Select $(1, 4, 10)$, $V = \{1, 2, 4\}$,
  $E = \{(1, 3, 15), (1, 5, 20), (2, 3, 15), (2, 4, 20), (2, 5, 20), (4, 5, 10)\}$
- Select $(4, 5, 10)$, $V = \{1, 2, 4, 5\}$,
  $E = \{(1, 3, 15), (1, 5, 20), (2, 3, 15), (2, 4, 20), (2, 5, 20), (5, 3, 30)\}$
- Select $(1, 3, 15)$, $V = \{1, 2, 4, 5, 3\}$,
  $E = \{(1, 5, 20), (2, 3, 15), (2, 4, 20), (2, 5, 20), (5, 3, 30)\}$

The Minimum Spanning Tree is visualized as...



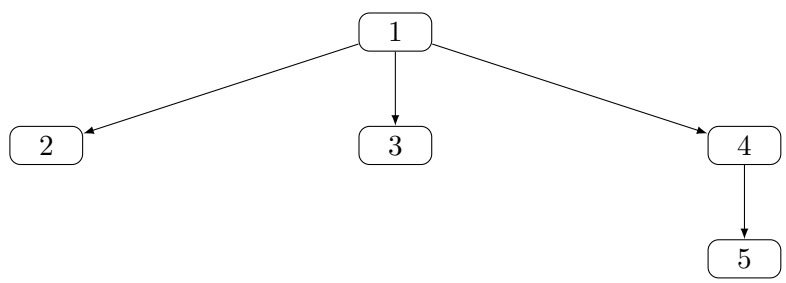
The bridge length is the sum of the weights of the selected edges, $10 + 10 + 10 + 15 = 45$.

(b) Illustrate the steps of Kruskal's algorithm using the graph above. Draw the Minimum Spanning Tree generated by your illustration. What is the length of the bridges?

Let (v, u, w) represented an edge from v to u with weight w. Let E be the set of available edges and V be the set of visited vertices.

- First, sort the edges by weight: $E_{sorted} = \{$
  $(1, 2, 10), (1, 4, 10), (4, 5, 10), (1, 3, 15), (2, 3, 15),$
  $(3, 4, 15), (1, 5, 20), (2, 4, 20), (2, 5, 20), (3, 5, 30)$
  $\}$
- Iterate through the list and select edges that contain an unexplored node.
  - Select $(1, 2, 10)$, $V = \{1, 2\}$
  - Select $(1, 4, 10)$, $V = \{1, 2, 4\}$
  - Select $(4, 5, 10)$, $V = \{1, 2, 4, 5\}$
  - Select $(1, 3, 15)$, $V = \{1, 2, 4, 5, 3\}$

The Minimum Spanning Tree is visualized as...



The bridge length is the sum of the weights of the selected edges, $10 + 10 + 10 + 15 = 45$.